

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Alumno

Francisco Diez.

Comisión

3

Grupo

472

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación 1

28 de junio de 2026

Índice

1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura	7
2.1. Diagrama de flujo de operaciones principales	7
2.2. Capturas de pantalla de la ejecución de ejemplos	8
3. Dificultades y conclusiones	10
3.1. Dificultades técnicas	10
3.2. Conclusiones	11
4. Links	11
5. Bibliografía	11

1. Marco Teórico

Todos los conceptos del marco teórico fueron proporcionados del material oficial de la materia y de la página oficial de Python (<https://docs.python.org/es/3.14/tutorial/index.html>).

- **Listas:** Las listas son secuencias mutables, ya que permiten modificar los valores de sus elementos, eliminar o añadir elementos. En ellas se pueden almacenar tantos elementos como se desee, incluso pueden existir listas vacías (sin elementos) y pueden contener elementos del mismo tipo o de distintos. Para definir las se utilizan corchetes [] y se separan los elementos mediante comas.

En este proyecto, la lista es la estructura principal utilizada para almacenar la información de los países. Cada elemento de la lista corresponde a un país representado mediante un diccionario. Gracias a esta estructura fue posible implementar funcionalidades como mostrar el listado completo, agregar nuevos países, modificar y eliminar registros, realizar búsquedas, aplicar filtros, ordenar los datos y calcular estadísticas. El uso de listas permitió recorrer todos los países mediante estructuras repetitivas (for), acceder a elementos específicos por su posición y manipular la colección de datos de manera dinámica durante la ejecución del programa.

- **Diccionarios:** Son una estructura de datos mutables que permite almacenar información mediante un sistema de pares clave-valor. Es decir, cada elemento que contiene tiene una clave única, asociada a un valor. Cada clave es única dentro del diccionario, no puede haber dos claves iguales. Los valores pueden ser de cualquier tipo de dato (números, cadenas, listas, otros diccionarios, etc).

En este proyecto, cada país se representa mediante un diccionario con las claves "nombre", "población", "superficie" y "continente". Esta organización permitió agrupar la información correspondiente a un país dentro de una única estructura de datos. Su uso facilitó la implementación de las distintas funcionalidades del sistema, ya que los datos podían accederse mediante sus claves, por ejemplo, `pais["nombre"]` o `pais["población"]`. Esto hizo que las operaciones de búsqueda, modificación, filtrado, ordenamiento y cálculo de estadísticas fueran más claras y fáciles de mantener.

- **Funciones:** Una función es un bloque de código reutilizable que realiza una tarea específica. Las funciones permiten organizar un programa en partes más pequeñas, evitando la repetición de código y facilitando su lectura, mantenimiento y reutilización. Pueden recibir datos de entrada mediante parámetros y, opcionalmente, devolver un resultado utilizando la instrucción `return`. En este proyecto se utilizaron funciones para dividir el programa en tareas específicas, siguiendo el principio de que cada función tenga una única responsabilidad. Por ejemplo, se implementaron funciones para cargar y guardar el archivo CSV, mostrar el menú, agregar, modificar y eliminar países, realizar búsquedas, aplicar filtros, ordenar la información y calcular estadísticas. Esta organización permitió desarrollar un código más ordenado y modular, facilitando tanto su mantenimiento como la incorporación de nuevas funcionalidades sin afectar el resto del programa.
- **Condicionales:** Las estructuras condicionales permiten ejecutar diferentes bloques de código en función del resultado de una condición lógica. Las instrucciones `if`, `elif` y `else` se utilizan para controlar el flujo de ejecución del programa, permitiendo tomar decisiones según determinadas situaciones.

En este proyecto, las estructuras condicionales se utilizaron en numerosas funciones para controlar el comportamiento del sistema. Por ejemplo, permitieron verificar si un país ya existía antes de agregarlo, comprobar si un país se encontraba en la lista antes de modificarlo o eliminarlo, validar que la población y la superficie fueran mayores que cero, controlar que los rangos de búsqueda fueran válidos y gestionar las distintas opciones seleccionadas por el usuario en los menús. El uso de estructuras condicionales permitió que el programa respondiera correctamente a las acciones del usuario, evitando errores y garantizando un funcionamiento más seguro y ordenado.

- **Ordenamiento:** Según la documentación oficial de Python, la función integrada `sorted()` permite ordenar los elementos de una colección sin modificar la original, devolviendo una nueva secuencia ordenada. Además, mediante el parámetro `key` es posible indicar el criterio de ordenamiento y, con el parámetro `reverse`, establecer si el orden será ascendente o descendente.

En este proyecto, la función `sorted()` se utilizó para ordenar la lista de países según distintos criterios: nombre, población y superficie. Para ello, se emplearon funciones **lambda** como criterio de ordenamiento, permitiendo seleccionar el atributo correspondiente de cada país. En el caso de la superficie, el sistema ofrece al usuario la posibilidad de elegir entre un orden ascendente o descendente mediante el parámetro `reverse`. El uso de esta función permitió presentar la información de manera organizada sin alterar la estructura original de la lista de países, facilitando la consulta y el análisis de los datos.

- **Estadísticas básicas:** Python dispone de diversas funciones integradas que permiten realizar operaciones sobre colecciones de datos de forma sencilla y eficiente. Entre ellas se encuentran `max()`, `min()` y `sum()`, utilizadas para obtener el valor máximo, el valor mínimo y la suma de un conjunto de elementos, respectivamente. Estas funciones también admiten el parámetro `key`, que permite indicar el criterio sobre el cual se realizará la operación.

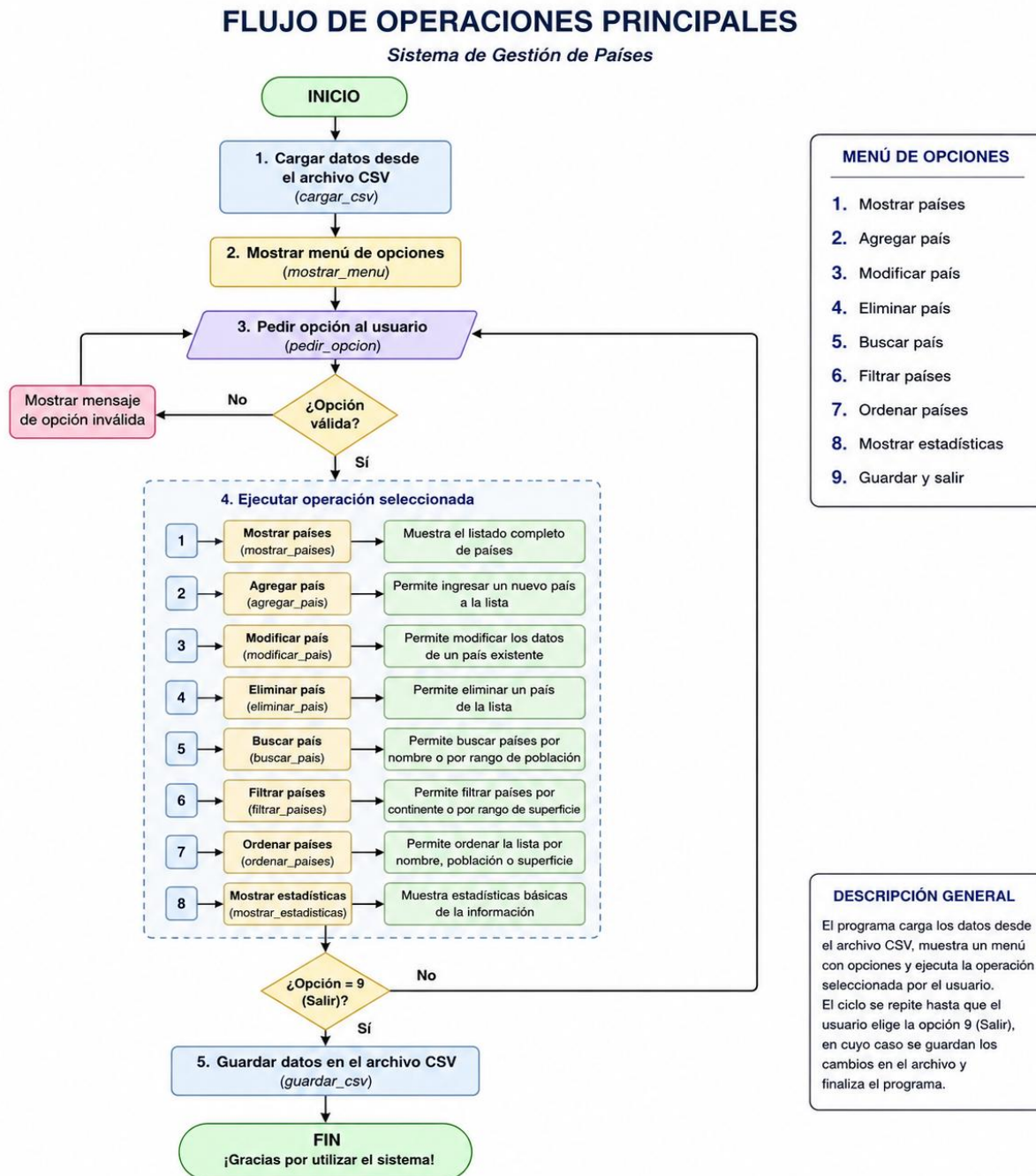
En este proyecto, estas funciones se emplearon para calcular estadísticas sobre los países almacenados en la lista. La función `max()` permitió identificar el país con mayor población, mientras que `min()` se utilizó para obtener el país con menor población. Por su parte, `sum()` permitió calcular el total de la población y de la superficie, a partir de los cuales se obtuvo el promedio de ambos valores. Además, mediante un diccionario se contabilizó la cantidad de países pertenecientes a cada continente. El uso de estas funciones facilitó la obtención de información estadística de manera clara, eficiente y con un código más simple que si los cálculos se hubieran realizado manualmente mediante recorridos y comparaciones.

- **Archivo CSV:** Un archivo **CSV** (*Comma-Separated Values*) es un formato de texto utilizado para almacenar datos tabulares, donde cada fila representa un registro y cada columna un campo. Python proporciona el módulo estándar `csv`, que facilita la lectura y escritura de este tipo de archivos mediante funciones especializadas.

En este proyecto, el archivo CSV se utilizó para almacenar de forma permanente la información de los países. Al iniciar el programa, los datos se cargan desde el archivo utilizando `csv.DictReader()`, convirtiendo cada fila en un diccionario que posteriormente se almacena en una lista. Al finalizar la ejecución del programa, los cambios realizados por el usuario se guardan nuevamente en el mismo archivo mediante `csv.DictWriter()`, asegurando la persistencia de la información. El uso del módulo `csv` permitió manipular los datos de manera sencilla y organizada, evitando que la información ingresada se perdiera al cerrar el programa.

2. Decisiones Técnicas y Arquitectura

2.1. Diagrama de flujo de operaciones principales



2.2. Capturas de pantalla de la ejecución de ejemplos

- Captura 1. Menú principal: la primera pantalla que ve el usuario.

```
=====
                        GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
3. Modificar país
4. Eliminar país
5. Buscar país
6. Filtrar países
7. Ordenar países
8. Estadísticas
9. Salir

Seleccione una opción:
```

- Captura 2. Mostrar países: opción número 1.

```
Seleccione una opción: 1
=====
                        LISTADO DE PAÍSES
=====
```

NOMBRE	POBLACIÓN	SUPERFICIE (km²)	CONTINENTE
Argentina	47.327.407	2.780.400	América
Brasil	212.583.750	8.515.767	América
Chile	19.603.733	756.102	América
Uruguay	3.426.260	176.215	América
Paraguay	6.823.445	406.752	América
Bolivia	12.413.315	1.098.581	América
Perú	34.038.757	1.285.216	América
Colombia	53.216.562	1.141.748	América
Venezuela	28.300.000	916.445	América
México	130.861.007	1.964.375	América

- Captura 3. Agregar un país: opción número 2.

```
Seleccione una opción: 2
=====
                        AGREGAR PAÍS
=====
Nombre: |
```


- Captura 4. Buscar un país: opción número 5.

```
=====
                        BUSCAR PAÍS
=====
Ingrese el nombre del país: Arg

=====
                        LISTADO DE PAÍSES
=====
NOMBRE                POBLACIÓN          SUPERFICIE (km²)    CONTINENTE
-----
Argentina             47.327.407         2.780.400           América
-----
Total de países: 1
```

- Captura 5. Filtrar: opción número 6.

```
=====
                        FILTRAR PAÍSES
=====
1. Filtrar por continente
2. Filtrar por rango de población
3. Filtrar por rango de superficie
4. Volver
Seleccione una opción: 1
```

- Captura 6. Estadísticas: opción número 8.

```
=====
                        ESTADÍSTICAS
=====
País con mayor población: India
Población: 1.438.069.596

País con menor población: Uruguay
Población: 3.426.260

Promedio de población: 157.802.489

Promedio de superficie: 1.671.207 km²

Cantidad de países por continente:
América: 10
Europa: 8
Asia: 6
África: 4
Oceanía: 2
```

- Captura 7. Salir: opción número 9 y finalización del programa.

```
=====
                        GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
3. Modificar país
4. Eliminar país
5. Buscar país
6. Filtrar países
7. Ordenar países
8. Estadísticas
9. Salir

Seleccione una opción: 9

¡Gracias por utilizar el sistema! Hasta luego.
```

3. Dificultades y conclusiones

3.1. Dificultades técnicas

Durante el desarrollo del proyecto surgieron distintos desafíos técnicos. Uno de los principales fue organizar correctamente la información utilizando listas y diccionarios, ya que fue necesario comprender cómo acceder, modificar y recorrer estas estructuras para implementar las distintas funcionalidades del sistema.

Otro desafío consistió en trabajar con archivos CSV para lograr la persistencia de los datos. Fue necesario aprender a utilizar el módulo csv para cargar la información al iniciar el programa y guardar los cambios realizados antes de finalizar la ejecución. Además, se implementó una ruta relativa mediante el módulo pathlib, permitiendo que el programa pudiera ejecutarse correctamente en distintos equipos sin depender de una ubicación específica del archivo.

También se presentaron dificultades al desarrollar los filtros por rango, los ordenamientos y el cálculo de estadísticas, ya que fue necesario aplicar correctamente funciones integradas de Python como sorted(), max(), min() y sum(), además de validar las entradas del usuario para evitar errores durante la ejecución.

Todos estos inconvenientes fueron resolviéndose mediante pruebas, depuración del código, revisión de la documentación oficial de Python y con la guía de la inteligencia artificial.

3.2. Conclusiones

La realización de este proyecto fue un total desafío y permitió integrar los conocimientos adquiridos durante la materia y aplicarlos en el desarrollo de un programa completo para la gestión de información. A lo largo del trabajo se reforzaron conceptos como el uso de funciones, listas, diccionarios, estructuras condicionales y repetitivas, manejo de archivos CSV y validación de datos.

Además de los aspectos técnicos, el proyecto permitió desarrollar habilidades relacionadas con la organización del código, la resolución de problemas y el pensamiento lógico para sortear todos los obstáculos. Como reflexión final, considero que esta experiencia resultó valiosa para comprender la importancia de diseñar programas de forma modular, escribir código claro y aplicar buenas prácticas de programación para facilitar su mantenimiento y evolución.

4. Links

- Video en YouTube: <https://youtu.be/UZmvPYv2Nc0>
- Repositorio en GitHub: https://github.com/frandiez-98/tpi_p1.git

5. Bibliografía

- Python Software Foundation. *Python 3 Documentation*. Disponible en: <https://docs.python.org/3/>
- Python Software Foundation. *Data Structures*. Disponible en: <https://docs.python.org/3/tutorial/datastructures.html>

- Python Software Foundation. *More Control Flow Tools*. Disponible en: <https://docs.python.org/3/tutorial/controlflow.html>
- Python Software Foundation. *Built-in Functions*. Disponible en: <https://docs.python.org/3/library/functions.html>
- Python Software Foundation. *csv — CSV File Reading and Writing*. Disponible en: <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. *pathlib — Object-oriented filesystem paths*. Disponible en: <https://docs.python.org/3/library/pathlib.html>
- Apuntes personales elaborados a partir del material de estudio y de las clases de la asignatura Programación I, correspondientes a la cursada.